COE223_3196 College of computer

2024/05/10 Department of computer science

Sorting 2D array using MIPs

Students names:

- SHATHA ALMUTAIRI 432205190 420
- AROUB ALFEHAID 431201578 422
- LEEN SULIMAN 431201407 430

Supervisor: Dr.Yasser A.

Table of Contents

Assembly Language	3
Introduction:	4
Project Management	5
Code	6
output	8
Conclusions	9
References	10
Appendix	10

Table of Figures

Figure 1: Gantt chart	5
Figure 2: Pert char	5
Figure 3:output of assembly program	8

Assembly Language

When we talk about programming languages the first thing that comes to our mind is languages like C, C++, Java, Python etc. But those languages hide the actual working i.e., that abstracts many things from users. But there is a language that really lies on basic concepts behind the programming or interaction between computer hardware.

What Is an Assembly Language?

An assembly language is a type of low-level programming language that is intended to communicate directly with a computer's hardware. Unlike machine language, which consists of binary and hexadecimal characters, assembly languages are designed to be readable by humans.

In assembly language, working with arrays and sorting them requires you to manage memory and data directly. In our project, we will demonstrate how you can work with a 2D array, sort it, and convert it to a 1D array.

The project showcases how to handle a 2D array in assembly language, sort it using algorithms applied to each row, and then convert it to a 1D array by sequentially copying each element. This requires precise control over memory and data manipulation, which are key skills in low-level programming such as assembly language.

Introduction:

In our project, we show how to work with a 2D array in assembly language. This includes sorting the 2D array and converting it into a 1D array. Our approach focuses on direct memory and data management, which is important for low-level programming.

Conditions for Use:

- Our solution is for systems where you can interact directly with hardware and manage memory directly using assembly language.
- We assume the 2D array fits within the available memory and can be accessed and manipulated directly.
-

Main Results:

- Sorted each row of the 2D array using a sorting algorithm.
- Converted the sorted 2D array into a 1D array by copying each element.
- Demonstrated precise control over memory and data, which is key in assembly language programming.

Report Structure:

- We start with an overview of assembly language and its importance for low-level programming.
- Then, we explain how we sort the 2D array and convert it to a 1D array.
- Finally, we present the results of our approach, showing the sorted array and the conversion.

Project Management

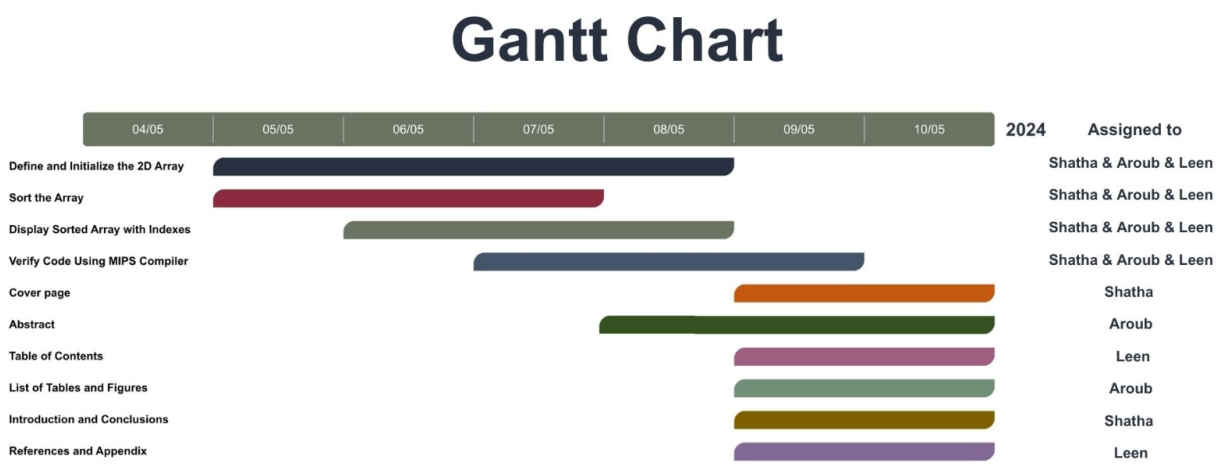

Figure 1: Gantt chart

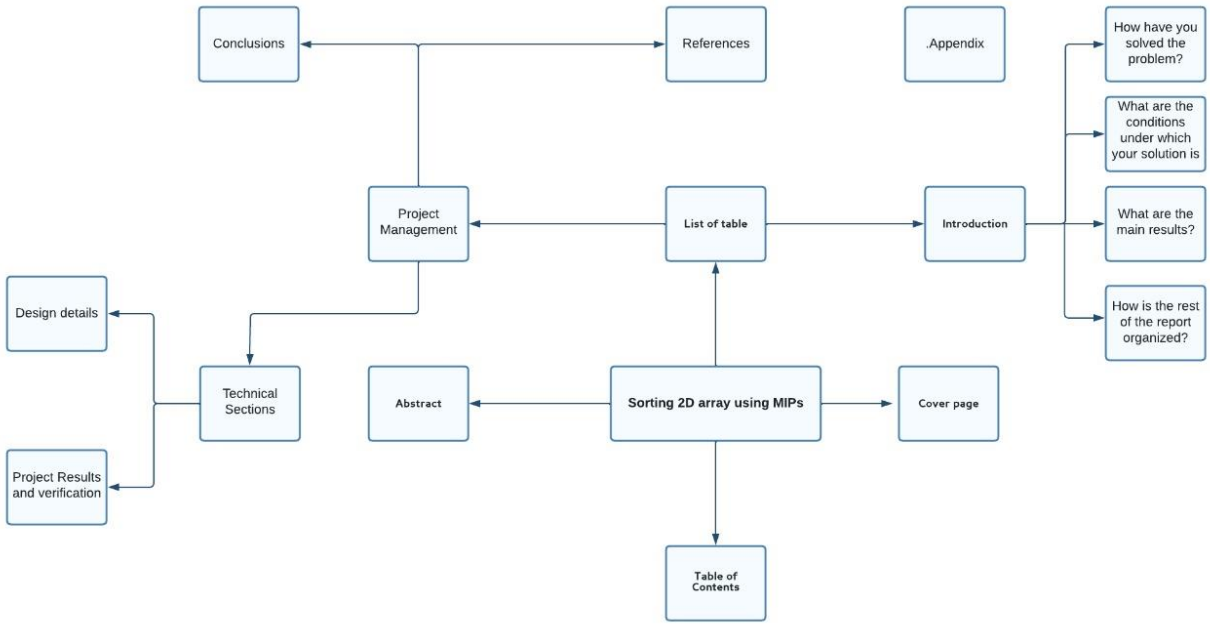

Figure 2: Pert char

Code

```
#The base address of array is 0x10000000
#$s0 = array base address, $s1 = i, $s2 = j
.data
str1 : .asciiz "Sorted Array"
str2 : .asciiz "Un sorted Array"
newline : .asciiz "\n"
space : .asciiz "  "
.text
main:
        li $s0, 0x10000000

        addi $s1, $0, 0     #i = 0
        addi $t1, $0, 8     #$t1 = 8

for1:
slt $t0, $s1, $t1    #i < 8
beq $t0, $0, done1                  #if not then exit

        addi $s2, $0, 0             #j = 0
for2:
slt $t0, $s2, $t1    #j < 8
beq $t0, $0, done2                  #if not then exit

sll $t0, $s1, 3                     #$t0 = i * 8
add $t0, $t0, $s2                   #$t0 = i * 8 + j
sll $t0, $t0, 2                     #$t0 = (i * 8 + j) * 4
sll $t2, $s2, 3                     #$t2 = j * 8
add $t2, $t2, $s1                   #$t2 = j * 8 + i
add $t0, $t0, $s0                   #address of array[i][j]
sw  $t2, 0($t0)                     #array[i][j] = $t2

addi $s2, $s2, 1                    #j = j + 1
        j for2                      #repeat
done2:
addi $s1, $s1, 1    #i = i + 1
        j for1                      #repeat
done1:
        addi $s1, $0, 0             #i = 0
        addi $t3, $0, 63            #$t3 = 63

la  $a0, str2                       #$a0 = str2
li $v0, 4
syscall                             #Print "Un sorted array"
la  $a0, newline                    #$a0 = new line
li $v0, 4
syscall                             #Print new line

# The rest of the program is on the other side
```

```
jal printArray

for3:
slt $t0, $s1, $t3    #i < 63
beq $t0, $0, done3                  #if not then exit

        addi $s2, $0, 0             #j = 0
for4:
slt $t0, $s2, $t3    #j < 63
beq $t0, $0, done4                  #if not then exit

sll $t0, $s2, 2                     #$t0 = j * 4
add $t0, $t0, $s0                   #address of array[j]
lw $t4, 0($t0)                      #$t4 = array[j]
lw $t5, 4($t0)                      #$t5 = array[j+1]

slt $t6, $t5, $t4                   # array[j+1] < array[j]
beq $t6, $0, if                     #if not then exit

# swap array[j], array[j+1]
        sw $t5, 0($t0)              #array[j] = $t5
        sw $t4, 4($t0)              #array[j+1] = $t4
if:
addi $s2, $s2, 1                    #j = j + 1
        j for4                      #repeat
done4:
addi $s1, $s1, 1                    #i = i + 1
        j for3                      #repeat
done3:
la  $a0, str1                       #$a0 = str1
li $v0, 4
syscall                             #Print "Un sorted array"
la  $a0, newline                    #$a0 = new line
li $v0, 4
syscall                             #Print new line

        jal printArray
li $v0, 10                          #exit
syscall

printArray:

        addi $s1, $0, 0             #i = 0
        addi $t1, $0, 8             #$t1 = 8

# The rest of the program is on the next pag
```

```
  for5:
slt $t0, $s1, $t1                    #i < 8
beq $t0, $0, done5                   #if not then exit

        addi $s2, $0, 0              #j = 0
for6:
slt $t0, $s2, $t1                    #j < 8
beq $t0, $0, done6                   #if not then exit

sll $t0, $s1, 3                      #$t0 = i * 8
add $t0, $t0, $s2                    #$t0 = i * 8 + j
sll $t0, $t0, 2                      #$t0 = (i * 8 + j) * 4
sll $t2, $s2, 3                      #$t2 = j * 8
add $t2, $t2, $s1                    #$t2 = j * 8 + i
add $t0, $t0, $s0                    #address of array[i][j]
lw  $a0, 0($t0)                      #$a0 = array[i][j]

li $v0, 1
syscall                              #Print the array[i][j]

la  $a0, space                       #$a0 = space
li $v0, 4
syscall                              #Print new line



addi $s2, $s2, 1                     #j = j + 1
        j for6                       #repeat
done6:

la  $a0, newline   #$a0 = new line
li $v0, 4
syscall                             #Print new line


addi $s1, $s1, 1   #i = i + 1
        j for5                      #repeat
done5:
la  $a0, newline   #$a0 = new line
li $v0, 4
syscall                            #Print new line


        jr $ra

    #end the program
```

output

```
Console

Un sorted Array
0  8  16  24  32  40  48  56
1  9  17  25  33  41  49  57
2  10  18  26  34  42  50  58
3  11  19  27  35  43  51  59
4  12  20  28  36  44  52  60
5  13  21  29  37  45  53  61
6  14  22  30  38  46  54  62
7  15  23  31  39  47  55  63

Sorted Array
0  1  2  3  4  5  6  7
8  9  10  11  12  13  14  15
16  17  18  19  20  21  22  23
24  25  26  27  28  29  30  31
32  33  34  35  36  37  38  39
40  41  42  43  44  45  46  47
48  49  50  51  52  53  54  55
56  57  58  59  60  61  62  63
```

Figure 3:output of assembly program

Conclusions

Our project achieved the goal of sorting a 2D array in assembly language and converting it into a 1D array. We used a sorting algorithm on each row of the 2D array and then copied the sorted elements into a 1D array, demonstrating efficient data management.

We faced some challenges, such as avoiding memory access issues and optimizing the sorting algorithm for speed. Despite these obstacles, the project showed how assembly language can handle low-level data manipulation tasks with precision and care. This highlights the importance of assembly language in tasks that require direct memory and data control.

References

Digital Design and Computer Architecture

Appendix

[1] https://shawnzhong.github.io/JsSpim/

[2] https://sourceforge.net/projects/spimsimulator/files/